

BACKSIDE SILUET

Reporte de la elaboración de el proyecto de Backside Siluet
enfocado en la dar a conocer la estructura del proyecto
Elaborado por Luis Fernando Mateo Vargas (Estudiante en
proceso de TI Desarrollo de software)

Reporte de
Programador

Reporte de Programador

1.1 Inicio

Para poder iniciar el proyecto, primero se planteó la problemática, la cual fue cómo nos daríamos a conocer. Una vez identificada la problemática, surgió la idea de desarrollar una página web para el gimnasio llamado Backside Siluet, que tiene como clientas principales a mujeres. Después de evaluar los pros y los contras de las tecnologías a utilizar, decidimos utilizar la tecnología de React y mongodb, ya que React ofrece a los posibles usuarios una mejor experiencia, y mongoDB nos ofrece gestionar una base de datos la cual nos ayuda para no ser necesarios los diagramas y agilizar el tiempo de programación.

1.2 Planteamiento

Durante el planteamiento del proyecto, se llevaron a cabo varias sesiones de discusión y planificación en las cuales se abordaron los detalles técnicos, incluyendo las tecnologías a utilizar, la arquitectura del sistema y las herramientas de desarrollo más adecuadas para asegurar que el proceso fuera eficiente y efectivo. Se puso especial atención en comprender las necesidades e intereses específicos de Backside Siluet, analizando sus objetivos comerciales y el tipo de información que deseaban comunicar a sus clientas.

En esta parte del proyecto, se dialogó con la gerente, licenciada Andrea Nataly Rojas Rendón, y se presentaron las sugerencias y requisitos que tenía en mente para su proyecto. Durante las sesiones, surgieron nuevos requerimientos y sugerencias. Una vez que el proyecto contó con la aprobación de la gerente, procedimos a la selección de las tecnologías a utilizar.

Después de seleccionar las tecnologías, que incluyeron React y Visual Studio Code y MongoDB, se planteó el calendario de entregables. Finalmente, se clarificaron los entregables del proyecto, especificando de manera clara y medible los contenidos y secciones que se esperaba desarrollar y presentar al culminar cada fase del proyecto, permitiendo un seguimiento efectivo del progreso. Al concluir este proceso de revisión y ajuste, el planteamiento del proyecto fue aprobado formalmente por todas las partes involucradas, llegando a un acuerdo claro y compartido que permitió dar inicio oficial al desarrollo del proyecto.

1.3 Ejecución

En la ejecución del proyecto, primero nos dedicamos a investigar las instalaciones del gimnasio Backside Siluet para entender mejor qué aspectos eran los más importantes a destacar en la página web. Esto nos permitió tener una visión clara de lo que queríamos lograr. Luego, comenzamos a crear los primeros bocetos del diseño, siempre teniendo en mente la identidad visual del gimnasio y las necesidades de sus clientas.

Durante esta fase, mantuvimos reuniones constantes con la licenciada Andrea Nataly Rojas Rendón para mostrarle nuestros avances y obtener su retroalimentación. Estas reuniones fueron muy útiles para asegurarnos de que estábamos en el camino correcto y para ajustar cualquier detalle necesario.

1.4 Control

Para mantener un control de las versiones y tener un respaldo de la información, nos dedicamos a investigar métodos de almacenamiento. Aunque hay varias opciones en el mercado, optamos por dos que nos resultaron muy cómodas no solo para el guardado del código, sino también para almacenar información relacionada, como imágenes, videos y documentos. Las opciones que elegimos fueron GitHub y Google Drive.

GitHub nos permite llevar un control detallado de las versiones del código, facilitando la colaboración en el equipo y asegurando que siempre tengamos acceso a las versiones anteriores del proyecto. Además, su sistema de control de versiones nos ayuda a rastrear los cambios y solucionar problemas de manera eficiente.

Por otro lado, **Google Drive** nos proporciona una solución sencilla y accesible para almacenar y compartir archivos multimedia e información adicional. Esto nos permite mantener toda la información organizada y disponible para todos los miembros del equipo en cualquier momento y lugar.

Al utilizar GitHub y Google Drive en conjunto, aseguramos un control efectivo del proyecto y una gestión adecuada de todos los recursos necesarios para el desarrollo del sitio web.

1.5 Cierre

Al finalizar nuestro proyecto de Backside Siluet, estamos contentos de haber alcanzado los objetivos que nos propusimos al inicio. La investigación detallada de las instalaciones y la colaboración constante con la licenciada Andrea Nataly Rojas Rendón nos permitieron crear una página web interactiva que refleja perfectamente la identidad del gimnasio y satisface las necesidades de sus clientas.

Gracias a la dedicación y el esfuerzo de todo el equipo de desarrollo, el nuevo sitio web de Backside Siluet se ha convertido en una herramienta poderosa que ayudará al gimnasio a alcanzar sus objetivos y a mejorar la experiencia de sus clientas. Estamos muy orgullosos del trabajo realizado y confiamos en que el sitio web contribuirá significativamente al éxito continuo de Backside Siluet.

2 Inicio

2.1 Investigación y Análisis

Al iniciar el proyecto para desarrollar la página web de Backside Siluet, comenzamos con una investigación detallada para entender a fondo los objetivos del gimnasio y sus necesidades específicas. Analizamos cómo se presenta la competencia y revisamos las tendencias actuales en diseño y tecnología web. Este análisis nos permitió asegurarnos de que nuestra propuesta no solo cumpliera con las mejores prácticas del sector, sino que también se alineara con las expectativas y los requerimientos del gimnasio.

2.2 Definición de Objetivos y Alcance

Trabajamos en estrecha colaboración con la licenciada Andrea Nataly Rojas Rendón para definir los objetivos claros del sitio web de Backside Siluet. Acordamos que la página web serviría como una plataforma interactiva para mostrar los servicios del gimnasio, permitiendo a las clientas conocer más sobre las instalaciones, programar clases y acceder a información relevante. También establecimos el alcance del proyecto, definiendo las funcionalidades principales que debía incluir el sitio web.

3 Planeación

3.1 Creación de Wireframes y Mockups

Para empezar con la planeación del sitio web de Backside Siluet, desarrollé wireframes que mostraban la disposición y estructura básica de la página. Estos wireframes nos ayudaron a visualizar cómo se organizarían los distintos elementos y se aseguraron de que todos los objetivos definidos se cumplieran adecuadamente. Luego, creé mockups estáticos que representaban de manera más detallada cómo se vería el diseño final de la página. Presenté estas maquetas a la licenciada Andrea Nataly Rojas Rendón para obtener su aprobación y asegurarme de que el diseño reflejara correctamente la visión del gimnasio antes de avanzar al desarrollo.



Ilustración 1 Wireframes

3.2 Selección de Tecnologías

Basado en la visión de Backside Siluet y los requisitos del proyecto, seleccioné las tecnologías más adecuadas para el desarrollo de la página web. Optamos por usar React para el frontend debido a su capacidad para crear interfaces interactivas y dinámicas, MongoDB como base de datos debido a su facilidad de uso y comodidad. Para el almacenamiento y gestión de datos, decidimos utilizar Google Drive, y GitHub para el control de versiones. Esta combinación de herramientas nos permitió asegurar un desarrollo eficiente y una gestión efectiva del contenido y del código

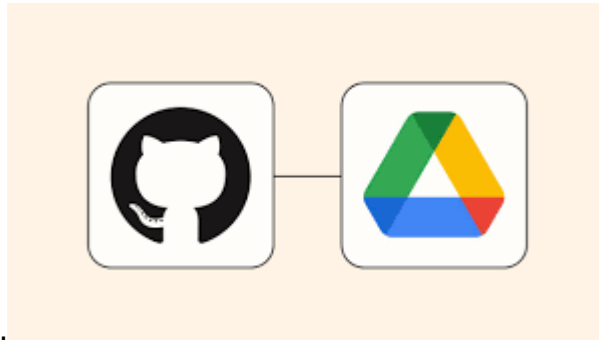


Ilustración 2: Google Drive, y GitHub

4 Ejecución

4.1 Diseño Visual

Me enfoqué en el diseño visual del sitio web de Backside Siluet, asegurándose de que la identidad del gimnasio se refleja en cada aspecto del diseño. Seleccioné colores, tipografías y elementos visuales que transmitieran la profesionalidad y la modernidad que Backside Siluet representa. Este diseño no solo buscaba ser estéticamente atractivo, sino también alineado con la imagen y el estilo del gimnasio para crear una experiencia agradable a las clientas.

SKETCH DESIGN	FINAL(12 th) SKETCH	Guys Berman
COMPANY APPLICATION	FIRST PAGE	Jonathan Martin
Homepage	Product list	Product+Details

Ilustración :3 Diseño visual

4.2 Desarrollo Frontend

Inicié el proceso de desarrollo transformando los diseños aprobados en código. Utilicé React para construir la interfaz de usuario, asegurándome de que fuera interactiva y dinámica. Complementé el trabajo con HTML y CSS para garantizar que el sitio fuera receptivo y se adaptara a diferentes dispositivos y tamaños de pantalla. Este enfoque permitió ofrecer una experiencia de usuario fluida y atractiva, cumpliendo con los estándares de diseño establecidos.

4.3 Desarrollo Backend

Mientras trabajaba en el frontend, también avancé en la implementación del backend utilizando JavaScript y MongoDB. Configuré el servidor y desarrollé la lógica necesaria para las funcionalidades interactivas del sitio. MongoDB se utilizó para manejar la base de datos, permitiendo el almacenamiento y recuperación eficiente de información relevante, como detalles de clases, horarios y perfiles de usuarios. Esta integración garantizó que el sitio web funcionara de manera cohesiva y efectiva, respaldando todas las operaciones requeridas.

4.4 Instalación de VS Code y Herramientas de Desarrollo

Aseguramos que todo el equipo de desarrollo tuviera instalado Visual Studio Code, junto con las extensiones necesarias para trabajar con React y MongoDB. Esto incluyó la configuración de herramientas de desarrollo que facilitarían el proceso de codificación y depuración, asegurando que el flujo de trabajo fuera lo más eficiente posible ([.visualstudio](https://code.visualstudio.com/))



Ilustración :4 Visual Studio Code

4.5 Revisión del Acuerdo de Licencia

Finalmente, revisamos y leímos cuidadosamente el acuerdo de licencia para asegurarnos de cumplir con todas las normativas y requisitos legales relacionados con las herramientas y tecnologías utilizadas en el proyecto. Esto nos permitió

proceder con confianza, sabiendo que estábamos operando dentro de los límites legales y respetando todas las políticas establecidas.

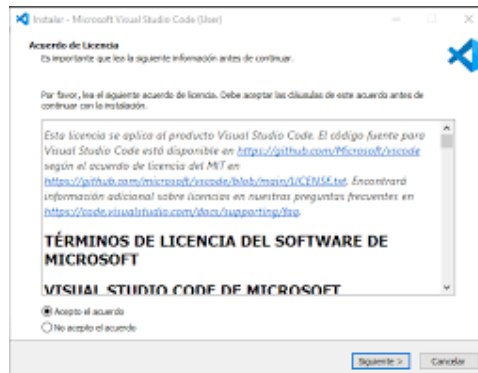


Ilustración 5: Licencia de Visual

4.6 Instalación de Node.js

En esta etapa del proyecto, llevamos a cabo la instalación de Node.js, una plataforma que nos permite ejecutar JavaScript en el servidor. A continuación, los pasos para instalarlo:

Descarga e Instalación:

Visita el sitio oficial de Node.js (<https://nodejs.org/>).

Descarga el instalador adecuado para tu sistema operativo (Windows, Linux o macOS).

Ejecuta el instalador descargado y sigue las instrucciones, acepta los términos y condiciones.



Ilustración 6: Node

4.6.1 Selección de Componentes

Durante la instalación, puedes elegir los componentes adicionales que necesitas. Por lo general, selecciona “Node.js” y “npm” (Node Package Manager). Haz clic en “Next” y espera a que finalice la instalación.



Ilustración 7: Node y NPM

4.6.2 Verificación

Abre la ventana de comandos (puedes presionar Win + R y escribir cmd).

Escribe `node -v` y presiona Enter.

Si ves la versión de Node.js, la instalación fue exitosa.

```
C:\WINDOWS\system32\cmd. x + v
Microsoft Windows [Versión 10.0.22631.3958]
(c) Microsoft Corporation. Todos los derechos reservados.

C:\Users\Luis Fernando>node -v
v20.12.1

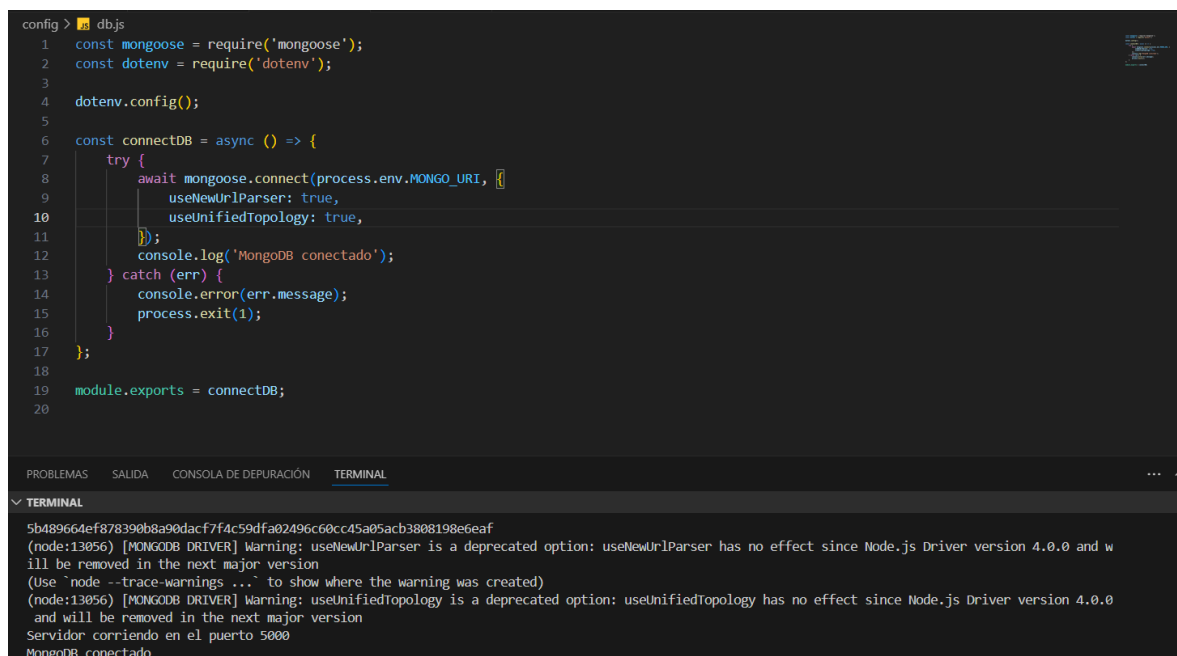
C:\Users\Luis Fernando>
```

Ilustración 8: Terminal Versión Node

4.7 Conexión de la Base de Datos con nuestra Web

En el desarrollo del proyecto para Backside Siluet, se realizó la configuración de la base de datos utilizando MongoDB para asegurar que nuestra web pudiera almacenar y gestionar eficientemente la información necesaria. Esto incluyó la configuración de una conexión segura entre la aplicación y la base de datos, la definición de las colecciones y campos relevantes, y la realización de pruebas exhaustivas para garantizar el correcto funcionamiento y la seguridad de los datos.

Para establecer la conexión entre la aplicación y MongoDB, se utilizó el siguiente código en Node.js, que emplea la biblioteca **mongoose** para interactuar con la base de datos:



```
config > db.js
1 const mongoose = require('mongoose');
2 const dotenv = require('dotenv');
3
4 dotenv.config();
5
6 const connectDB = async () => {
7   try {
8     await mongoose.connect(process.env.MONGO_URI, {
9       useNewUrlParser: true,
10      useUnifiedTopology: true,
11    });
12    console.log('MongoDB conectado');
13   } catch (err) {
14     console.error(err.message);
15     process.exit(1);
16   }
17 };
18
19 module.exports = connectDB;
20
```

PROBLEMAS SALIDA CONSOLA DE DEPURACIÓN TERMINAL

5b489664ef878390b8a90daf7f4c59dfa02496c60cc45a05acb3808198e6eaf
(node:13056) [MONGODB DRIVER] Warning: useNewUrlParser is a deprecated option: useNewUrlParser has no effect since Node.js Driver version 4.0.0 and will be removed in the next major version
(Use `node --trace-warnings ...` to show where the warning was created)
(node:13056) [MONGODB DRIVER] Warning: useUnifiedTopology is a deprecated option: useUnifiedTopology has no effect since Node.js Driver version 4.0.0 and will be removed in the next major version
Servidor corriendo en el puerto 5000
MongoDB conectado

Ilustración 9: Conexión con Base de datos Mongodb (Local)

4.7.1 Finalización de Instalación

Se completó la instalación y configuración de las herramientas necesarias para el desarrollo del proyecto de Backside Siluet. Después de establecer Visual Studio Code (VS Code) como el entorno de desarrollo principal, node como el servidor de react y configurar MongoDB como la base de datos, se realizó una verificación exhaustiva de su funcionalidad para asegurarse de que todo estuviera en óptimas condiciones.

4.8 Elección de Paleta de Colores

En la fase de diseño del proyecto para Backside Siluet, se eligió una paleta de colores cuidadosamente seleccionada para reflejar la identidad del gimnasio y crear una experiencia visual atractiva. Los colores seleccionados son los siguientes:

- **#c850c0**: Un tono vibrante de rosa que aporta energía y dinamismo, ideal para destacar elementos clave y llamar la atención.
- **#ffffff**: El blanco, utilizado para proporcionar un contraste limpio y garantizar que el contenido sea legible y visualmente accesible.
- **#9f3e9f**: Un tono de rosa más profundo que añade sofisticación y profundidad al diseño, complementando el color principal.
- **rgba(85, 85, 85, 1)**: Un gris neutro que ofrece equilibrio y una base sólida para el diseño, permitiendo que otros colores resalten.
- **#e19cdc**: Un rosa suave que proporciona un toque delicado y elegante, ideal para fondos o detalles sutiles.
- **#ff69b4**: Conocido como "hot pink", este color intenso se utiliza para acentos y llamadas a la acción, añadiendo un toque llamativo.
- **#6a0dad**: Un morado intenso que agrega un sentido de lujo y exclusividad, perfecto para áreas destacadas y secciones importantes.
- **#d4a5ff**: Un tono lila claro que complementa el esquema de colores, proporcionando un contraste suave y añadiendo una sensación de frescura.

Esta paleta de colores fue elegida para asegurar que el diseño del sitio web no solo sea visualmente atractivo, sino también coherente con la imagen de marca de Backside Siluet. La combinación de estos colores permite crear una interfaz amigable y envolvente, que atraerá y retendrá a las visitantes del gimnasio, además de facilitar una navegación fluida y una experiencia general positiva.



Ilustración 10: Paleta de Colores

4.9 Pagina principal Inicio / Home

La página principal de "Backside Siluet" está pensada para dar una cálida bienvenida y ofrecer una visión completa de lo que el gimnasio tiene para ofrecer, la página web se componen por varias secciones las cuales brinda una mejor distribución de la información y ayuda a dar una mayor claridad a la hora de compartir la información .

4.9.1 Encabezado

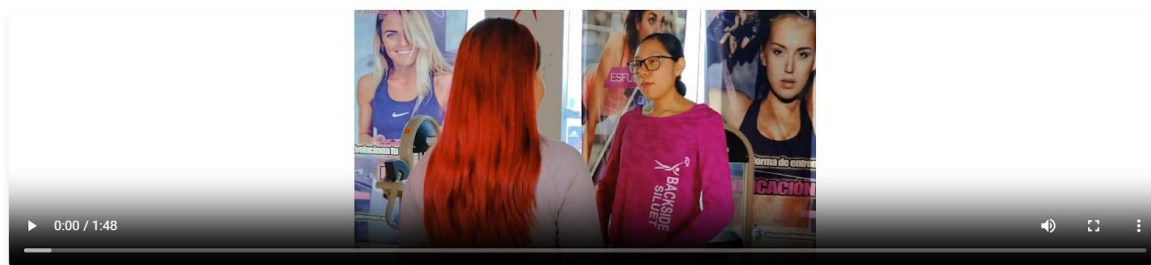
En la parte superior, tenemos el logotipo del gimnasio junto con un menú de navegación claro y sencillo. Desde aquí te puedes dirigir a las otras pantallas tales como son, "Inicio", "Instalaciones", "Nosotros", "Danza Aérea" y "Inicio Sesión". Además, hay un botón llamativo que dice "Únete al club" el cual te mostrará la pantalla de cotización de las membresías, perfecto para quienes quieren inscribirse rápidamente. También hay íconos de redes sociales (Facebook, Instagram, WhatsApp y TikTok) para que puedas seguirnos y mantenerte al tanto de todas nuestras novedades.



Ilustración 11: Barra de Navegación de Home

4.9.2 Sección de Bienvenida

Aquí encontrarás un video de bienvenida que se reproduce en bucle, mostrando una pequeña muestra de lo que es "Backside Siluet". Al lado del video, te damos la bienvenida con un mensaje que destaca nuestro objetivo principal: ayudarte a alcanzar tus metas de acondicionamiento físico y aumentar tu masa muscular.



Bienvenida

¡Bienvenidos a Backside Siluet! Somos un gimnasio enfocado en el entrenamiento de mujeres. Nuestro objetivo es ayudarte a alcanzar tus metas de acondicionamiento físico y aumentar tu masa muscular. ¡Únete a nuestra comunidad y comienza tu viaje hacia una vida más saludable y activa!

Ilustración 12: Sesión Bienvenida

4.9.3 Sección Hero

En esta sección, una imagen impactante junto con un botón que dice "Inscríbete al club" te invita a formar parte de nuestra comunidad. Queremos captar tu atención y motivarte a tomar acción para mejorar tu salud y bienestar.



Inscríbete al club

Ilustración 13: Imagen Principal

4.9.4 Beneficios del Ejercicio

Aquí te mostramos algunos de los beneficios del ejercicio, especialmente diseñados para mejorar la condición y la salud de las mujeres :

Mejora de la Salud Cardiovascular: Fortalece tu corazón y mejora la circulación.

Fortalecimiento del Sistema Musculoesquelético: Fortalece tus huesos y músculos.

Mejora del Estado de Ánimo y Reducción del Estrés: Libera endorfinas que te hacen sentir bien.

Mantenimiento y Control del Peso Corporal: Ayuda a quemar calorías y mantenerte en forma.

Beneficios Transformadores del Ejercicio para las Mujeres

	Mejora de la Salud Cardiovascular El ejercicio regular, especialmente el cardiovascular, fortalece el corazón y mejora la circulación sanguínea, lo que reduce el riesgo de enfermedades cardíacas. También se mejora la capacidad pulmonar y se regula la presión arterial.
	Fortalecimiento del Sistema Musculoesquelético La práctica de ejercicios de resistencia y fuerza ayuda a aumentar la densidad ósea y fortalecer los músculos. Esto es particularmente importante para las mujeres, ya que reduce el riesgo de osteoporosis y lesiones.
	Mejora del Estado de Ánimo y Reducción del Estrés El ejercicio físico libera endorfinas, conocidas como las hormonas de la felicidad, que mejoran el estado de ánimo y reducen los niveles de estrés y ansiedad. También ayuda a combatir la depresión y mejorar la calidad del sueño.
	Mantenimiento y Control del Peso Corporal El ejercicio regular es esencial para quemar calorías y mantener un peso corporal saludable. Ayuda a aumentar el metabolismo ya regular el apetito, lo que facilita el control del peso a largo plazo.

Ilustración 14: Beneficios

También te contamos por qué "Backside Siluet" es la mejor opción: somos exclusivos para mujeres, nos enfocamos en una transformación integral, mejoramos tu autoestima y contamos con una comunidad de apoyo.

¿Por qué elegirnos?

- Exclusividad para Mujeres: Disfruta de un entorno cómodo y acogedor, diseñado específicamente para ti.
- Transformación Integral: Nos enfocamos en tu bienestar físico, mental y emocional para una transformación completa.
- Mejora de la Autoestima: Te ayudamos a sentirte segura y orgullosa de ti misma a través de programas individualizados.
- Comunidad de Apoyo: Forma parte de una comunidad solidaria y motivadora que te impulsa a alcanzar tus metas.
- Área de Vitalidad: Combinamos ejercicio, bienestar y desarrollo personal para ofrecerte una experiencia única.

Ilustración 15: Porque elegirnos

4.9.5 Programas

En esta parte damos a conocer los diferentes programas de entrenamiento los cuales son:

1. **Oxidativo:** Para quemar grasas con ejercicios de diferentes intensidades.
2. **Hipertrofia:** Para aumentar la masa muscular en la parte inferior del cuerpo.
3. **Glucolítico:** Trabajo anaeróbico de alta intensidad.
4. **Clásico:** Tonificación muscular y mejora de capacidades físicas funcionales.



Oxidativo

Su principal objetivo es la oxidación de grasas, manteniendo una frecuencia cardíaca base, intercalando ejercicios de alta, moderada y baja intensidad.



Hipertrofia

Su principal objetivo es el incremento muscular en el tren inferior; glúteos, cuádriceps, femorales.



glucolítico

Su principal objetivo es el trabajo anaeróbico de alta intensidad para utilizar los depósitos glucolíticos y terminar con una oxidación de O_2 .



• Clásico

Enfocado en la tonificación muscular de grupos musculares superiores, inferiores o mixtos. Prevención de lesiones y mejora de capacidades físicas funcionales.

Ilustración 16: Programas

4.9.6 Testimonios

Aquí puedes leer lo que nuestras clientas tienen que decir. Sus historias y experiencias positivas muestran cómo "Backside Siluet" ha tenido un impacto real en sus vidas.

Testimonios



Me encuentro muy feliz ya gusto con los avances y resultados que noto día con día, no solo físico sino el cambio emocional ya que también las personas a mi alrededor lo pueden notar. El destino de tiempo y dedicación a mi persona es fundamental para mi bienestar en general ya que siendo madre muchas veces nos negamos ese privilegio que es tener una vida saludable.

-Karla Alvarado



Me ha encantado la experiencia que me ofrece Backside Siluet ya que al ser un gimnasio exclusivo para mujeres lo hace diferente pero sobre todo especial. Me gusta que los entrenadores estén al tanto de que realice adecuadamente los ejercicios, pero sobre todo me gusta más el ambiente y la buena convivencia que hay. Llevo ya 8 meses en Backside y un punto a destacar es que las rutinas no se me hacen aburridas ya que cada mes es algo distinto. El hacer ejercicio me mantiene más activado durante mi rutina diaria sobre todo mejora mi estado de ánimo gracias Backside por cambiar mi vida...

- Itzel Gamboa



Tome la decisión de hacer ejercicio por salud, siento que es importante como mujeres tener la iniciativa de cuidar nuestro cuerpo, de no quedar en el intento y demostrar que somos capaces de lograr objetivos para nosotras mismas. Me gusta mucho el ambiente que hay, las rutinas son muy buenas me encantan, las entrenadoras siempre están pendientes de nosotras para cuidarnos que hagamos bien los ejercicios y sobre todo darnos ese ánimo 🌟.

- Noemí Cortez



Hola soy eleaneth porras tengo 29 años de edad y llevo más de dos años aquí en el gimnasio de Backside. La verdad a mí me fascina mucho venir a entrenar aquí, se ha vuelto parte importante de mi día a día, me gusta venir aquí por qué las entrenadoras te ponen la rutina y están ahí en todo momento para apoyarte y guiarte. Además que cuentan con una nutrióloga que te ayuda a obtener tus resultados y las metas que te propongas. Se podría decir que Backside sin duda algún cambio mi vida por completo básicamente me ha ayudado a tener más seguridad en mí, más fuerza y mayor autoestima, emocionalmente me siento muy feliz por qué he conocido a gente maravillosa que ahora son parte importante de mi vida. Lo recomiendo muchísimo para todas aquellas mujeres que quieran hacer un cambio en su vida.

Eleaneth Porras

Ilustración 17: Testimonios

4.9.7 Pie de Página

En el pie de página, encontrarás nuestra información de contacto, enlaces rápidos a otras secciones del sitio y nuestros perfiles de redes sociales. Además, te recordamos nuestros derechos de autor.



Contacto
Dirección: 3 norte #206 centro
Tehuacán, Puebla.
Teléfono: +1 234 567 890
Correo electrónico:
backside.siluet@gmail.com

Enlaces Rápidos

- Inicio
- Instructores
- Precios
- Noticias
- Danza aérea

Síguenos



© 2024 Club de Fitness. Todos los derechos reservados.

Ilustración 18: Pie de página de Inicio

4.10 Pantalla Instalaciones

La pantalla de "Instalaciones" de "Backside Siluet" es una ventana a todo lo que nuestro gimnasio tiene para ofrecer.

4.10.1 Encabezado

Al igual que la pantalla de inicio al entrar, te recibe nuestro encabezado con el logo de "Backside Siluet" a la izquierda, que representa nuestra dedicación y pasión por el fitness. Justo al lado, un menú de navegación sencillo y directo te permite acceder

a las secciones de "Inicio", "Suscripciones" y "Nosotros". Estos enlaces te llevarán a conocer más sobre nuestros servicios y nuestra misión.



Ilustración 19: Barra de navegación de Instalaciones



Ilustración 20: Código de Barra de navegación

4.10.2 Sección de Bienvenida

En el corazón de la página, un gran banner te da la bienvenida con una imagen inspiradora y un mensaje motivador: "Contamos con nuestro único y propio sistema de entrenamiento Backside". Queremos que sientas desde el primer momento que estás en el lugar adecuado para alcanzar tus objetivos fitness.



Ilustración 21: Banner de bienvenida de Instalaciones

```
1 <main className="principal-instalaciones">
2   <section className="banner">
3     <img src={IMAGEN_BANNER} alt="Banner" />
4     <div className="texto-banner">
5       <h1>Contamos con nuestro único y propio sistema de entrenamiento Backside</h1>
6     </div>
```

Ilustración 22: Código de Banner

4.10.3 Instalaciones Especializadas

Nuestro gimnasio está diseñado pensando en ti, con áreas y equipos especializados que destacan por su calidad y eficacia contamos con tamos también con programa de alimentación y monitoreo de tu progreso

```

1 <section className="seccion-principal">
2   <div className="cuadrícula-instalaciones">
3     <Instalacion
4       titulo="Aparatos especializados para mujeres "
5       imagenes=[[EspecializadoMujer, EspecializadoMujer1, EspecializadoMujer2]]
6       descripcion="Entrenamiento diseñado para aumentar la masa muscular mediante ejercicios de resistencia y fuerza."
7     />
8     <Instalacion
9       titulo="Enfoque integral "
10      imagenes=[[EnfoqueIntegral, EnfoqueIntegral1, EnfoqueIntegral2, EnfoqueIntegral3, EnfoqueIntegral4]]
11      descripcion="Sesiones intensivas enfocadas en mejorar la capacidad anaeróbica y la resistencia a la fatiga."
12    />
13    <Instalacion
14      titulo="Evaluaciones iniciales y seguimiento mensual "
15      imagenes=[[Evaluacion1, Evaluacion2, Evaluacion3, Evaluacion4]]
16      descripcion="Programa de entrenamiento aeróbico para mejorar la eficiencia cardiovascular y la quema de grasa."
17    />
18    <Instalacion
19      titulo="Programa de alimentación saludable"
20      imagenes=[[Alimentacion, Alimentacion2, Alimentacion3]]
21      descripcion="Métodos tradicionales de entrenamiento para un desarrollo equilibrado de fuerza y resistencia."
22    />
23    <Instalacion
24      titulo="Rutinas de alta intensidad "
25      imagenes=[[AltaIntencidad1, AltaIntencidad2, AltaIntencidad3]]
26      descripcion="Equipo y rutinas diseñadas específicamente para el fortalecimiento y tonificación del cuerpo femenino."
27    />
28    <Instalacion
29      titulo="Rutinas glúteos / cuádriceps "
30      imagenes=[[RutinaCuadricep, RutinaCuadricep1, RutinaCuadricep2, RutinaCuadricep3, RutinaCuadricep4, RutinaCuadricep5]]
31      descripcion="Equipo y rutinas diseñadas específicamente para el fortalecimiento y tonificación del cuerpo femenino."
32    />
33    <Instalacion
34      titulo="Rutinas superiores / funcionales "
35      imagenes=[[RutinaFuncionales, RutinaFuncionales1, RutinaFuncionales2, RutinaFuncionales3]]
36      descripcion="Equipo y rutinas diseñadas específicamente para el fortalecimiento y tonificación del cuerpo femenino."
37    />
38  </div>
39 </section>

```

Ilustración 23: Código de los carruseles de Instalaciones

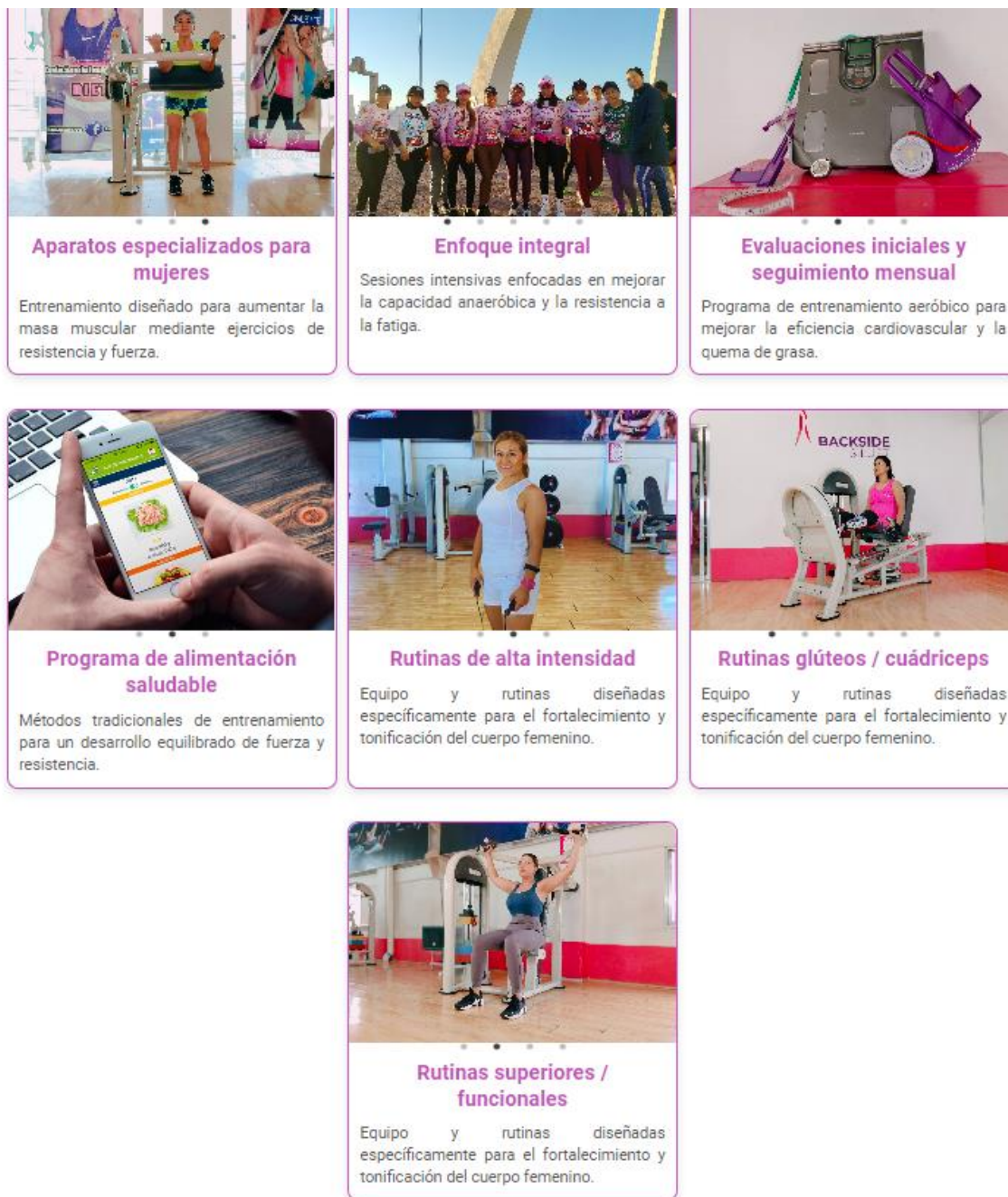


Ilustración 24: Carruseles de Instalaciones

4.11 Pantalla Sobre Nosotros

En la sección "Inicio" de la página principal de Backside Siluet, se presenta a los usuarios una bienvenida cálida y un resumen de los servicios y la misión del centro. Esta pantalla tiene como objetivo captar la atención de los visitantes y ofrecerles una visión general de lo que pueden esperar al unirse a la comunidad Backside

Siluet. La página se estructura de manera que la navegación sea sencilla y atractiva, facilitando el acceso a las diferentes secciones del sitio.

4.11.1: Vista de la sección "Nosotros" | Backside Siluet

La disposición de los elementos visuales y textuales está diseñada para proporcionar una experiencia de usuario intuitiva y enriquecedora. La sección incluye:

- **Encabezado:** Contiene el logotipo de Backside Siluet y un menú de navegación que enlaza a las principales secciones del sitio.
- **Sección de Bienvenida:** Un mensaje motivacional acompañado de una imagen representativa, invitando a los usuarios a unirse y explorar más sobre las suscripciones y pruebas de bienvenida.
- **Misión y Visión:** Secciones dedicadas a explicar la misión y visión del centro, acompañadas de imágenes relacionadas.
- **Actividades y Comunidad:** Descripción de las actividades que se realizan y cómo fomentan la comunidad.
- **Colaboradoras:** Presentación del equipo de entrenadoras a través de un carrusel de imágenes.
- **Eventos en la Comunidad:** Información sobre los eventos organizados para fomentar la comunidad.
- **Sistema de Entrenamiento:** Descripción del exclusivo sistema de entrenamiento Backside Siluet.

Esta estructura asegura que los usuarios tengan acceso rápido y claro a la información más relevante sobre Backside Siluet, facilitando su toma de decisiones y promoviendo la participación activa en la comunidad.



Ilustración 25: Pantalla de Nosotros

4.11.2 Código / Nosotros

A continuación se presenta el código correspondiente a la sección "Nosotros" en la página principal de Backside Siluet. Este código React muestra cómo se estructuran y estilizan los elementos visuales y textuales.

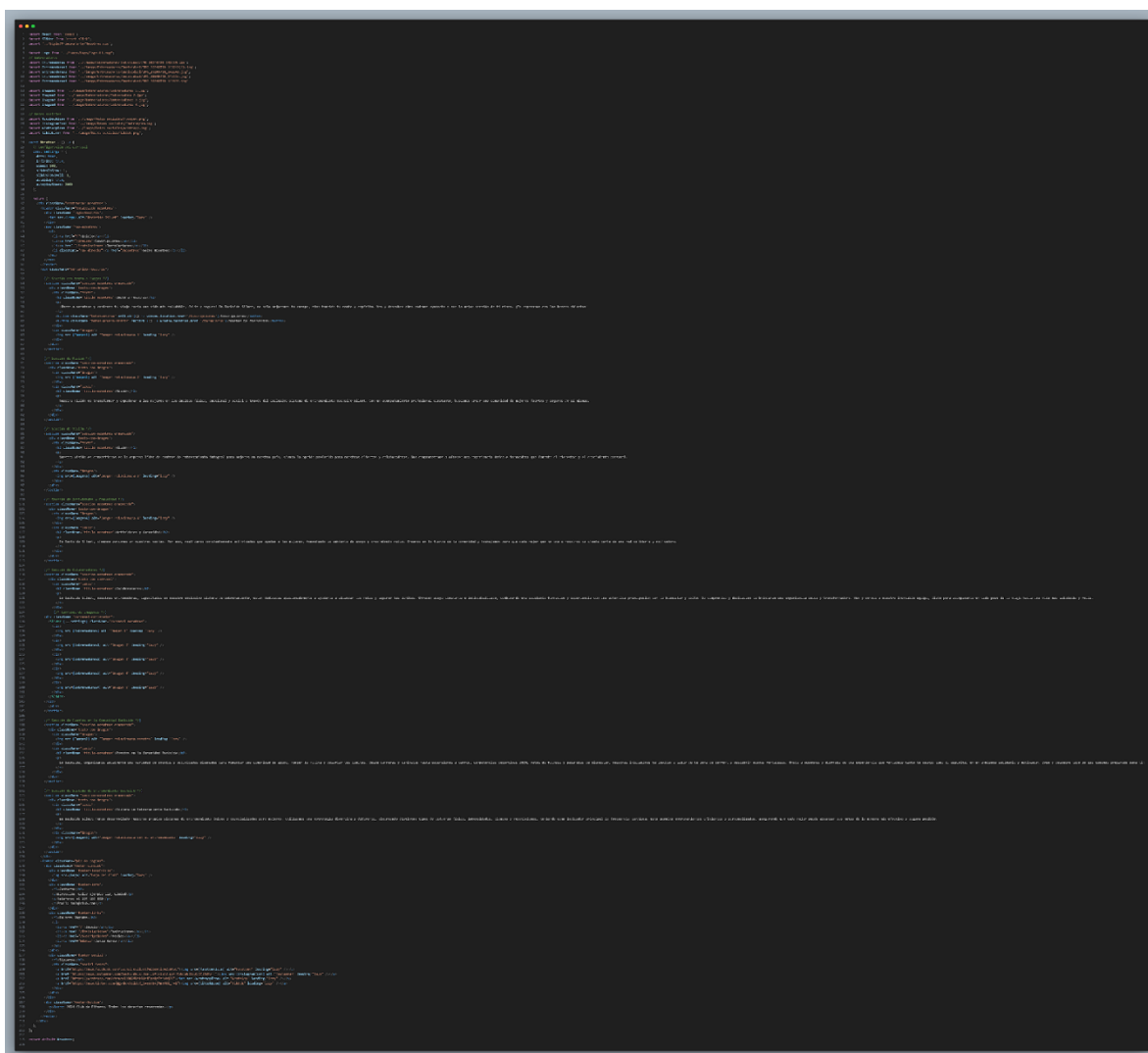


Ilustración 26: Código de Nosotros

En este bloque de código, cada sección del contenido "Nosotros" está cuidadosamente estructurada utilizando componentes de React. Las imágenes se cargan de manera diferida (loading="lazy") para mejorar el rendimiento de la página, y se utiliza la biblioteca react-slick para crear un carrusel de imágenes en la sección de colaboradoras. Las clases CSS definidas en ../Style/PrimeraParte/Nosotros.css se aplican para estilizar y posicionar los elementos de manera consistente y atractiva.

4.12: Suscripciones

En la sección "Suscripciones" de la página de Backside Siluet, ofrecemos una visión clara y organizada de los diferentes planes de suscripción disponibles para los clientes. Esta pantalla está diseñada para mostrar de manera concisa los beneficios y precios de cada nivel de suscripción, facilitando a las usuarias la elección del plan que mejor se adapte a sus necesidades.



Ilustración 27: Precios de las membresías

4.12.1: Vista de la sección "Suscripciones" | Backside Siluet

La disposición de los elementos en esta sección incluye:

- **Encabezado:** Un encabezado que contiene el logotipo de la empresa y la barra de navegación.
- **Contenido Principal:** Una lista de los diferentes planes de suscripción con sus respectivos precios y beneficios.
- **Pie de Página:** Información sobre derechos de autor.

4.4.12.2 Código / Suscripciones

A continuación se presenta el código correspondiente a la sección "Suscripciones". Este código escrito con tecnología React muestra cómo se estructura y presenta la información de los planes de suscripción.

```

1 import React from 'react';
2 import '../Style/PrimeraParte/Suscripciones.css';
3 import Logo from '../image/Logo/logo-01.png';
4
5
6 const suscripciones = [
7   {
8     nivel: 'Oro',
9     precio: 600,
10    precioEstudiante: 500,
11    beneficios: [
12      'Acceso a nutrimidad: app para dieta',
13      'Acceso a rookmotion: App para sensor de seguimiento',
14      'Dieta y educación',
15      'Entrenamiento',
16      'Seguimiento de peso y medidas',
17      'Expediente de resultados mensuales'
18    ]
19  },
20   {
21     nivel: 'Plata',
22     precio: 550,
23     precioEstudiante: 480,
24     beneficios: [
25       'Acceso a rookmotion: App para sensor de seguimiento',
26       'Seguimiento de peso y medidas',
27       'Entrenamiento',
28       'Expediente de resultados mensuales'
29     ]
30   },
31   {
32     nivel: 'Bronce',
33     precio: 450,
34     precioEstudiante: 400,
35     beneficios: [
36       'Entrenamiento',
37       'Expediente de resultados mensuales',
38       'Medida Mensual'
39     ]
40   }
41 ];
42
43 function Suscripciones() {
44   return (
45     <div className="contenedor"> {/* Cambiado 'container' por 'contenedor' */}
46       <header className="encabezadoSuscripciones"> {/* Cambiado 'headerSuscripciones' por 'encabezadoSuscripciones' */}
47         <div className="logo"> {/* Cambiado 'logoS' por 'logo' */}
48           <img src={Logo} alt="Backside Siljet" />
49         </div>
50         <h1>Suscripciones</h1>
51         <nav>
52           <ul>
53             <li><a href="/">Inicio</a></li>
54             <li><a href="/instalaciones">Instalaciones</a></li>
55             <li><a href="/Nosotros">Nosotros</a></li>
56           </ul>
57         </nav>
58       </header>
59
60       <section className="contenidoSuperior"> {/* Cambiado 'contentSup' por 'contenidoSuperior' */}
61         {suscripciones.map((plan, index) => (
62           <div className="plan" key={index}>
63             <h2>{plan.nivel}</h2>
64             <p><strong>Precio:</strong> ${plan.precio}</p>
65             <p><strong>Precio Estudiante:</strong> ${plan.precioEstudiante}</p>
66             <ul>
67               {plan.beneficios.map((beneficio, i) => (
68                 <li key={i}>{beneficio}</li>
69               ))}
70             </ul>
71             <button className="botonSuscribir">Suscribirse</button> //Cambiado 'subscribe-btn' por 'botonSuscribir' */}
72           </div>
73         ))}
74       </section>
75
76       <footer className="pieDePagina"> {/* Cambiado 'footer' por 'pieDePagina' */}
77         <p>&copy; 2024 Gym para Mujeres. Todos los derechos reservados.</p>
78       </footer>
79     </div>
80   );
81 }
82
83 export default Suscripciones;
84

```

4.13 Formulario de Registro

El formulario de registro es una parte esencial de la interfaz de usuario ya que permite a los nuevos posibles clientes acceder a una promoción de visitas gratis, esto con el fin de que pueda conocer las instalaciones, solamente proporcionando información sobre su nombre, metas e información de contacto relacionadas con su experiencia en gimnasios. Utilizamos React para manejar el estado del formulario y EmailJS para enviar los datos recopilados por correo electrónico.



Ilustración 29: Pantalla de Formulario

4.13.1 Código / Formulario de Registro

importaciones y Configuración Inicial:

- React, useState: Biblioteca de React y el hook useState para manejar el estado del formulario.
- emailjs-com: Biblioteca para enviar correos electrónicos.
- Formulario.css: Archivo CSS para los estilos del formulario.
- logo: Importación del logo para ser utilizado en el formulario.

Estado del Formulario:

- Utilizamos useState para manejar los datos del formulario (formData).

Funciones de Manejo de Cambios y Envío:

- handleChange: Actualiza el estado del formulario a medida que el usuario ingresa datos.

- `handleSubmit`: Maneja el evento de envío del formulario. Define los parámetros del correo y utiliza `emailjs.send` para enviar los datos. Luego, resetea el formulario y muestra una alerta de éxito o error.

Renderizado del Formulario:

- El formulario incluye campos para nombre, email, teléfono, edad, experiencia previa en gimnasios, y metas y objetivos.
- Los datos ingresados se envían a través de EmailJS cuando el usuario hace clic en el botón de enviar.
- Se incluye un botón para regresar a la página anterior.

```

1 import React, { useState } from 'react';
2 import emailjs from 'emailjs-com';
3 import '../Style/PrimerParte/Formulario.css';
4 import logo from '../Image/Logo/logo-01.png';
5
6 function Formulario() {
7   const [formData, setFormData] = useState({
8     nombre: '',
9     email: '',
10    telefono: '',
11    edad: '',
12    experiencia: '',
13    metas: '',
14  });
15
16  const handleChange = (e) => {
17    setFormData({ ...formData, [e.target.name]: e.target.value });
18  };
19
20  const handleSubmit = (e) => {
21    e.preventDefault();
22
23    // Define el contenido del correo
24    const emailParams = {
25      from_name: formData.nombre,
26      from_email: formData.email,
27      message: `
28        Nombre: ${formData.nombre}\n
29        Email: ${formData.email}\n
30        Teléfono: ${formData.telefono}\n
31        Edad: ${formData.edad}\n
32        Experiencia previa: ${formData.experiencia}\n
33        Metas: ${formData.metas}
34      `;
35    };
36
37    // Envía el correo usando EmailJS
38    emailjs.send(
39      'service_b05kcsq', // Reemplaza con tu Service ID
40      'template_by47z52', // Reemplaza con tu Template ID
41      emailParams,
42      'CH0vg3Gxp3leGxFOQ' // Reemplaza con tu Public Key
43    )
44      .then((response) => {
45        alert('Datos enviados con éxito!');
46        setFormData({
47          nombre: '',
48          email: '',
49          telefono: '',
50          edad: '',
51          experiencia: '',
52          metas: '',
53        });
54      })
55      .catch((err) => {
56        alert('Error al enviar los datos, inténtelo de nuevo.');
```

Ilustración 30: Código de Formulario

4.14 Colaboradoras

En la sección "Inicio Colaboradoras" de la página de Backside Siluet, se proporciona a las colaboradoras un acceso rápido y sencillo a las principales funciones necesarias para su trabajo diario. Esta pantalla tiene como objetivo facilitar la gestión y acceso a diferentes secciones del sistema, permitiendo a las colaboradoras realizar sus tareas de manera eficiente.

4.14.1: Vista de la sección "Inicio Colaboradoras" | Backside Siluet

La disposición de los elementos en esta sección está diseñada para ser intuitiva y funcional. La página incluye:

- **Encabezado:** Un título claro y conciso que indica la sección actual.
- **Botones de Navegación:** Un conjunto de botones que enlazan a las principales funciones disponibles para las colaboradoras, como la búsqueda de clientes, capacitación, registro de nuevos clientes y generación de reportes.

Esta estructura asegura que las colaboradoras puedan acceder rápidamente a las herramientas y recursos necesarios para desempeñar sus funciones de manera efectiva.

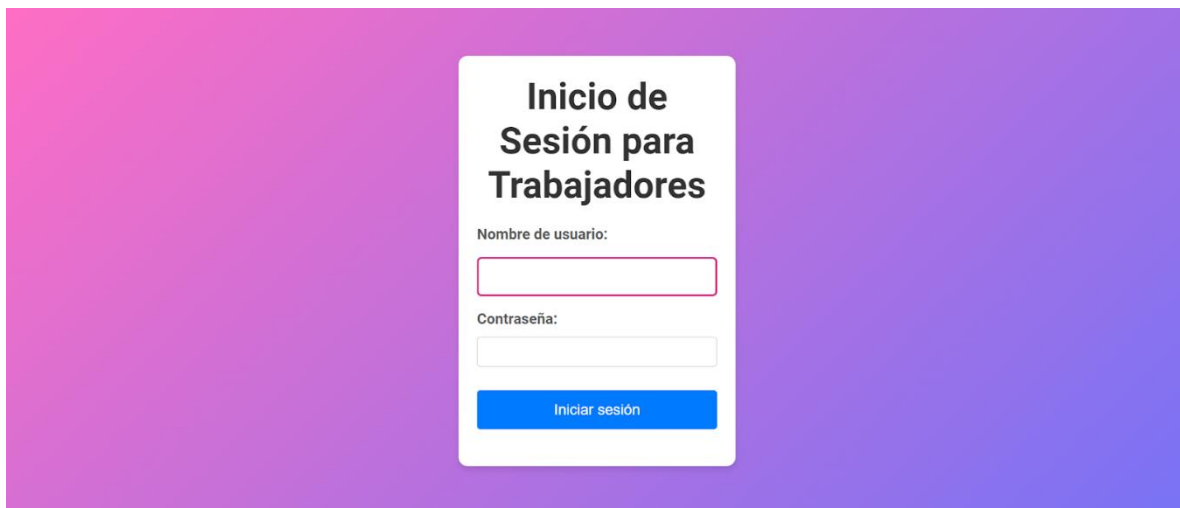


Ilustración 31: Pantalla de inicio de sesión Trabajadores

4.15 Código / Inicio Colaboradoras

A continuación, se presenta el código correspondiente a la sección "Inicio Colaboradoras". Este código React muestra cómo se estructuran y estilizan los elementos visuales y funcionales.

Inicio Colaboradoras

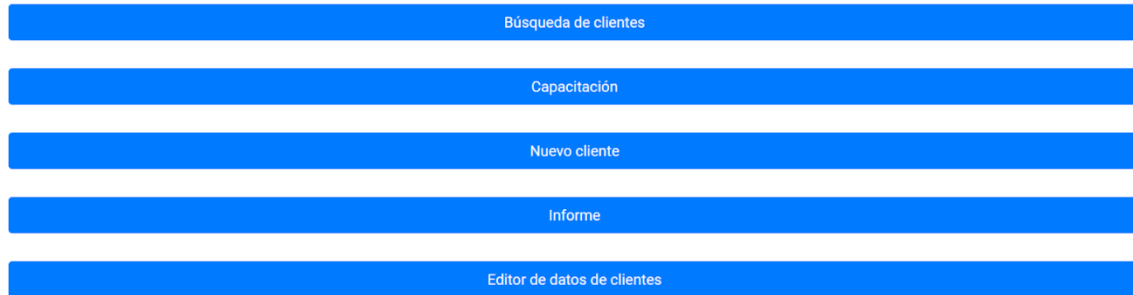


Ilustración 32: Panel de control de colaboradoras

En este bloque de código, la estructura de la pantalla "Inicio Colaboradoras" se compone de un contenedor principal con una clase CSS específica (inicio-colaboradoras). Dentro del contenedor, hay un título y una serie de botones que utilizan el componente Link de react-router-dom para la navegación. Cada botón tiene una clase CSS (boton) que se define en el archivo de estilos correspondiente (../Style/Colaboradores/InicioColaboradoras.css) para asegurar una apariencia coherente y profesional.

```
1 import React from 'react';
2 import { Link } from 'react-router-dom';
3 import '../Style/Colaboradores/InicioColaboradoras .css';
4
5 const InicioColaboradoras = () => {
6   return (
7     <div className="inicio-colaboradoras">
8       <h1>Inicio Colaboradoras</h1>
9       <div className="botones">
10        <Link to="/BusquedaCliente" className="boton">Búsqueda de cliente</Link>
11        <Link to="/Capacitacion" className="boton">Capacitación</Link>
12        <Link to="/RegistroCliente" className="boton">Nuevo Cliente</Link>
13        <Link to="/Reporte" className="boton">Reporte</Link>
14      </div>
15    </div>
16  );
17 }
18
19 export default InicioColaboradoras;
20
```

Ilustración 33: Código de panel de control

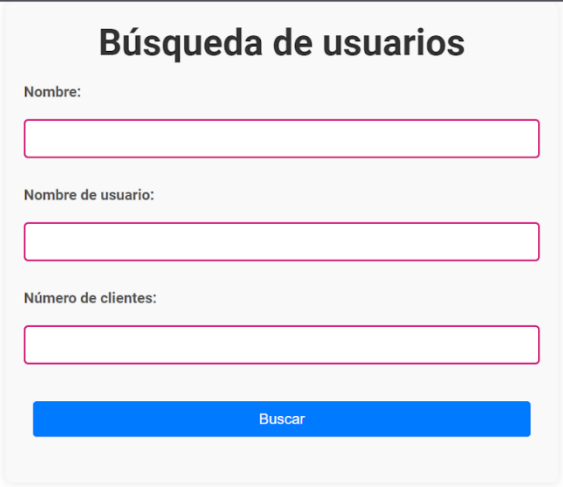
4.16 Búsqueda de Cliente

En la sección "Búsqueda de Cliente" de la página de Backside Siluet, se proporciona una interfaz para buscar usuarios en función de diferentes criterios. Esta pantalla está diseñada para permitir a las colaboradoras realizar búsquedas eficientes y obtener resultados relevantes rápidamente.

4.16.1: Vista de la sección "Búsqueda de Cliente" | Backside Siluet

La disposición de los elementos en esta sección está estructurada para facilitar la búsqueda de clientes con un formulario intuitivo y una visualización clara de los resultados. La página incluye:

- **Encabezado:** Un título que indica la funcionalidad de la página.
- **Formulario de Búsqueda:** Campos de entrada para buscar clientes por nombre, nombre de usuario o número de cliente, y un botón para enviar la consulta.
- **Mensajes:** Mensajes que informan a la usuaria sobre el estado de la búsqueda.
- **Resultados de la Búsqueda:** Una lista de resultados que muestra la información relevante de los usuarios encontrados.



El panel de búsqueda de usuarios tiene un título "Búsqueda de usuarios" en negrita. Debajo del título hay tres campos de entrada con etiquetas "Nombre:", "Nombre de usuario:" y "Número de clientes:". Cada campo tiene un borde de color magenta. Debajo de los campos hay un botón azul con el texto "Buscar".

Ilustración 34: Panel de Buscar Usuario

4.16.2 Código / Búsqueda de Cliente

A continuación se presenta el código correspondiente a la sección "Búsqueda de Cliente". Este código React muestra cómo se estructura y gestiona el formulario de búsqueda y los resultados.

En este bloque de código, se define el componente `BusquedaCliente`, que incluye:

- **Estados Locales:** Para almacenar los valores de los campos de entrada, los resultados de la búsqueda y los mensajes.
- **Manejador de Envío:** Una función `handleSubmit` que realiza una solicitud a la API con los parámetros de búsqueda y maneja la respuesta, actualizando el estado con los resultados o mensajes de error.
- **Formulario:** Un formulario con campos de entrada para nombre, nombre de usuario y número de cliente, y un botón de búsqueda.
- **Resultados:** Una sección que muestra los resultados de la búsqueda, con información detallada de cada usuario encontrado.

Este código asegura una búsqueda efectiva y una presentación clara de los datos relevantes para las colaboradoras.

```

1 import React, { useState } from 'react';
2 import '../Style/Colaboradores/BusquedaCliente.css';
3
4 const BusquedaCliente = () => {
5   const [nombre, setNombre] = useState('');
6   const [nombreDeUsuario, setNombreDeUsuario] = useState('');
7   const [numeroDeCliente, setNumeroDeCliente] = useState('');
8   const [resultados, setResultados] = useState([]);
9   const [mensaje, setMensaje] = useState('');
10
11   const handleSubmit = async (e) => {
12     e.preventDefault();
13
14     try {
15       const params = new URLSearchParams();
16       if (nombre) params.append('nombre', nombre);
17       if (nombreDeUsuario) params.append('nombreDeUsuario', nombreDeUsuario);
18       if (numeroDeCliente) params.append('numeroDeCliente', numeroDeCliente);
19
20       const response = await fetch('http://localhost:5000/api/usuarios/buscar?${params.toString()}');
21
22       if (response.ok) {
23         const data = await response.json();
24         if (data.length === 0) {
25           setMensaje('No se encontraron usuarios');
26         } else {
27           setMensaje('');
28           setResultados(data);
29         }
30       } else {
31         const errorText = await response.text();
32         console.error('Error en la búsqueda:', errorText);
33         setMensaje('Error en la búsqueda');
34       }
35     } catch (err) {
36       console.error('Error en la búsqueda:', err);
37       setMensaje('Error en la búsqueda');
38     }
39   };
40
41   return (
42     <div className="busqueda-cliente">
43       <h1>Búsqueda de Usuario</h1>
44       <form onSubmit={handleSubmit} className="busqueda-formulario">
45         <div className="campo">
46           <label>Nombre:</label>
47           <input
48             type="text"
49             value={nombre}
50             onChange={(e) => setNombre(e.target.value)}
51           />
52         </div>
53         <div className="campo">
54           <label>Nombre de Usuario:</label>
55           <input
56             type="text"
57             value={nombreDeUsuario}
58             onChange={(e) => setNombreDeUsuario(e.target.value)}
59           />
60         </div>
61         <div className="campo">
62           <label>Número de Cliente:</label>
63           <input
64             type="text"
65             value={numeroDeCliente}
66             onChange={(e) => setNumeroDeCliente(e.target.value)}
67           />
68         </div>
69         <button type="submit" className="boton">Buscar</button>
70       </form>
71       {mensaje && <p>{mensaje}</p>}
72       <div className="resultados">
73         {resultados.map((usuario) => (
74           <div key={usuario._id} className="resultado">
75             <p><strong>Nombre:</strong> {usuario.nombre}</p>
76             <p><strong>Nombre de Usuario:</strong> {usuario.nombreDeUsuario}</p>
77             <p><strong>Número de Cliente:</strong> {usuario.numeroDeCliente}</p>
78             <p><strong>Contraseña:</strong> {usuario.contrasena}</p>
79           </div>
80         ))}
81       </div>
82     </div>
83   );
84 };
85
86 export default BusquedaCliente;
87

```

Ilustración 35: Código de Panel de Búsqueda

4.17 Registro de Usuario

En la sección "Registro de Usuario" de la página de Backside Siluet, se ofrece una interfaz completa para registrar nuevos usuarios en el sistema. Esta pantalla está diseñada para permitir a las colaboradoras registrar clientes con toda la información necesaria de manera clara y efectiva.

4.17.1: Vista de la sección "Registro de Usuario" | Backside Siluet

La disposición de los elementos en esta sección incluye:

- **Encabezado:** Un título que indica la funcionalidad de la página.
- **Formulario de Registro:** Campos de entrada para registrar toda la información del usuario, incluyendo nombre de usuario, contraseña, número de cliente, y otros datos personales.
- **Mensajes de Estado:** Mensajes que informan sobre el éxito o error en el registro.
- **Botón de Envío:** Un botón para enviar el formulario de registro.



Ilustración 36: Panel de Registro Usuario

4.17.2 Código / Registro de Usuario

A continuación se presenta el código correspondiente a la sección "Registro de Usuario". Este código React muestra cómo se estructura y gestiona el formulario de registro y las notificaciones de estado.

En este bloque de código, se define el componente Registro, que incluye:

- **Estados Locales:** Para manejar los datos del formulario, mensajes de error y éxito.
- **Manejador de Cambios:** La función handleChange actualiza los valores del formulario en el estado.
- **Manejador de Envío:** La función handleSubmit envía los datos del formulario al servidor, maneja la respuesta y redirige a la página de inicio si el registro es exitoso.
- **Formulario:** Un formulario con campos de entrada para toda la información del usuario, incluyendo nombre de usuario, contraseña, número de cliente, y más.
- **Mensajes de Estado:** Sección para mostrar mensajes de error o éxito según el resultado del registro.

Este código asegura una experiencia de registro eficiente y clara para las colaboradoras, con manejo adecuado de errores y confirmaciones de éxito.

4.18 Edición de Usuarios

La pantalla de edición de usuarios permite a los administradores modificar la información de los usuarios registrados en el sistema. Esta funcionalidad es crucial para mantener los datos de los usuarios actualizados y gestionar cambios en sus perfiles. Utilizamos React para manejar el estado del formulario y las operaciones de CRUD (Crear, Leer, Actualizar, Borrar) con la API del backend.

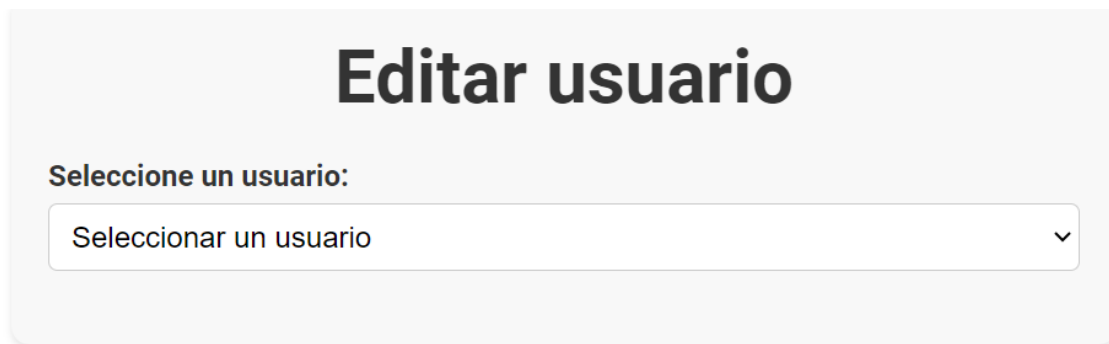
The image shows a web interface for editing a user. It features a light gray background with a white rounded rectangle in the center. At the top of the rectangle, the text 'Editar usuario' is displayed in a large, bold, black font. Below this, the text 'Seleccione un usuario:' is shown in a smaller, bold, black font. Underneath is a white dropdown menu with a thin gray border. The dropdown menu contains the text 'Seleccionar un usuario' and a small downward-pointing arrow icon on the right side.

Ilustración 38: Panel de Editar Usuario

4.18.1 Código / Edición de Usuarios

Importaciones y Configuración Inicial:

- React, useState, useEffect: Biblioteca de React y los hooks useState y useEffect para manejar el estado y los efectos secundarios.

- EditarUsuario.css: Archivo CSS para los estilos de la pantalla de edición de usuarios.

Estado del Componente:

- usuarios: Lista de usuarios obtenida del backend.
- selectedUsuario: Usuario seleccionado para editar.
- formData: Datos del formulario que se utilizan para actualizar la información del usuario.

Efecto para Obtener Usuarios:

- useEffect: Obtiene la lista de usuarios desde el backend cuando el componente se monta.

Funciones de Manejo de Cambios y Envío:

- handleUsuarioChange: Actualiza el estado del usuario seleccionado y los datos del formulario cuando se selecciona un usuario de la lista.
- handleInputChange: Actualiza el estado del formulario a medida que el usuario ingresa datos.
- handleSubmit: Maneja el evento de envío del formulario. Envía los datos actualizados del usuario al backend para actualizar su información.

Renderizado del Formulario:

- Incluye un selector para elegir un usuario de la lista.
- Muestra el formulario de edición con los datos del usuario seleccionado.
- Cada campo del formulario está vinculado a los datos del estado formData.
- Botón de envío para actualizar la información del usuario.



Ilustración 39: Código de Editar Usuario

4.19 Backend

En la sección "Backend" del proyecto de Backside Siluet, hemos desarrollado una infraestructura robusta utilizando JavaScript puro para la implementación del servidor y la lógica de negocio. El backend está diseñado para manejar eficientemente las solicitudes del frontend, interactuar con la base de datos, y proporcionar las funcionalidades necesarias para el funcionamiento de la aplicación.

La decisión de utilizar JavaScript para el desarrollo del backend se basa en su capacidad para proporcionar un entorno de desarrollo unificado, aprovechando la misma sintaxis y herramientas que se usan en el frontend. Este enfoque permite una mayor cohesión y eficiencia en el desarrollo del proyecto.

4.19.1 Backend / Base de datos MongoDB

Para conectar nuestro backend con MongoDB, utilizamos Mongoose, una biblioteca de modelado de datos que simplifica el trabajo con bases de datos MongoDB en aplicaciones Node.js. La configuración de la conexión se realiza a través de un módulo separado que gestiona el proceso de conexión y maneja posibles errores durante la conexión.

4.19.1.2 Código / Conexión a MongoDB

El siguiente código muestra la implementación del módulo de conexión a MongoDB usando Mongoose. Este módulo se encarga de establecer una conexión con la base de datos utilizando las credenciales y configuraciones especificadas en un archivo de entorno.

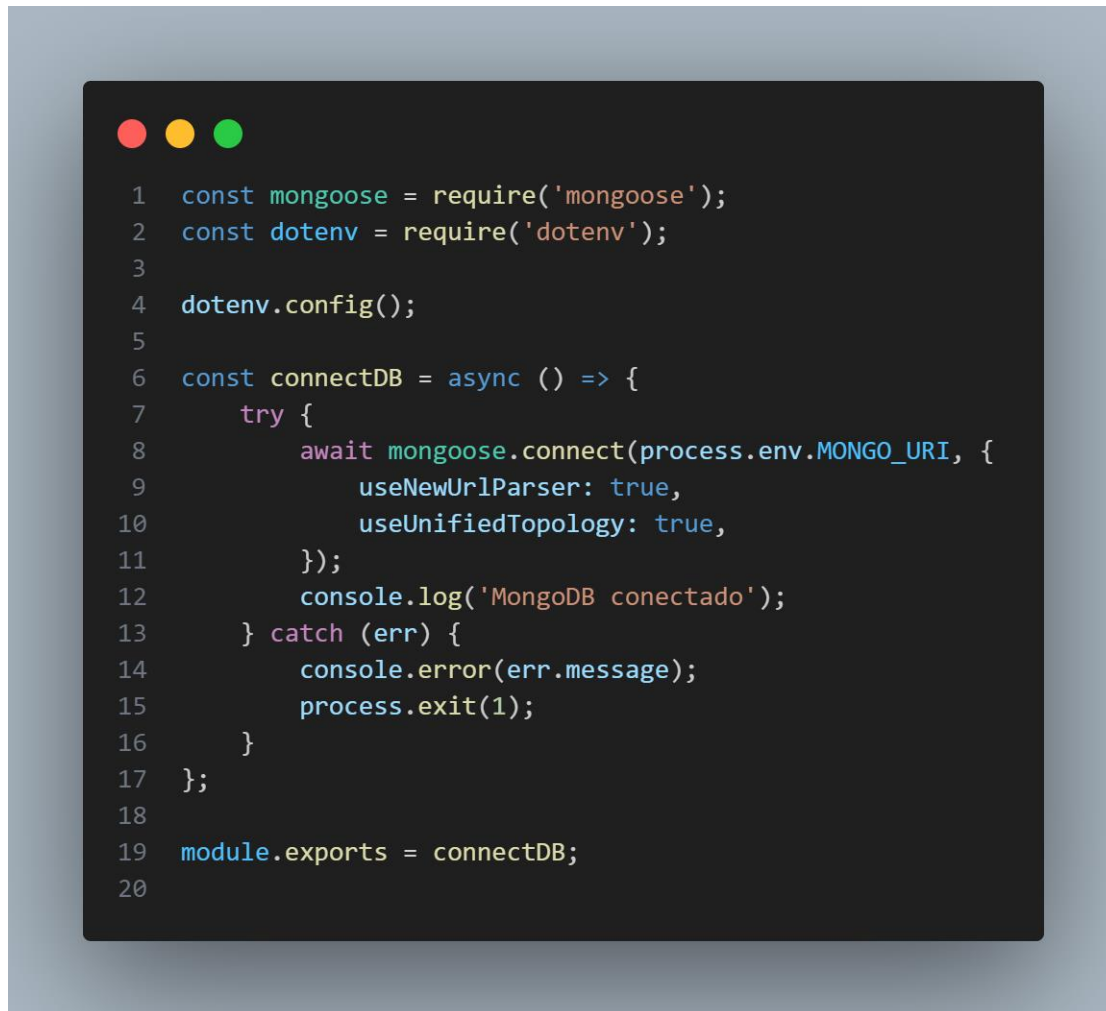


Ilustración 40: Código de Conexión a base de datos

4.19.1.3 Descripción del Código

Importación de Módulos:

mongoose: Se utiliza para interactuar con MongoDB de manera sencilla. dotenv: Permite cargar variables de entorno desde un archivo .env.

Configuración de Variables de Entorno:

dotenv.config(): Carga las variables de entorno desde el archivo .env donde se especifica la URI de conexión a MongoDB.

Función de Conexión:

connectDB: Una función asíncrona que intenta conectar con la base de datos MongoDB utilizando la URI proporcionada en las variables de entorno. Utiliza await para manejar la conexión de manera asíncrona.

Manejo de Errores:

En caso de error durante la conexión, el error se muestra en la consola y el proceso se cierra con un código de error 1.

Exportación:

module.exports = connectDB: Exporta la función connectDB para que pueda ser utilizada en otras partes del backend, permitiendo la conexión a la base de datos en el inicio de la aplicación.

4.19.1.4 Importancia

Este módulo es fundamental para establecer la comunicación entre el backend y la base de datos MongoDB, asegurando que los datos se puedan almacenar y recuperar de manera eficiente. La implementación garantiza que cualquier problema de conexión se maneje adecuadamente, permitiendo una gestión adecuada de errores y una mayor estabilidad en el sistema.

4.19.2 Backend / Middleware de Autenticación

En esta sección, se describe la implementación del middleware de autenticación en el backend del proyecto. El middleware se encarga de verificar la validez del token JWT (JSON Web Token) enviado en las solicitudes, asegurando que solo los usuarios autenticados puedan acceder a ciertas rutas protegidas.

La autenticación basada en tokens JWT es una técnica común para proteger las rutas en aplicaciones web. Un token JWT se genera al iniciar sesión y se incluye en las cabeceras de las solicitudes posteriores. El middleware de autenticación verifica este token y permite o deniega el acceso a las rutas protegidas según su validez.

4.19.2.1 Código / Middleware de Autenticación

El siguiente código muestra la implementación del middleware de autenticación en Node.js usando el paquete jsonwebtoken. Este middleware verifica la presencia y validez de un token JWT en las solicitudes entrantes.



```

1  // middleware/auth.js
2  const jwt = require('jsonwebtoken');
3
4  module.exports = function (req, res, next) {
5      const token = req.header('x-auth-token');
6      if (!token) {
7          return res.status(401).json({ msg: 'No hay token, permiso no válido' });
8      }
9
10     try {
11         const decoded = jwt.verify(token, process.env.JWT_SECRET);
12         req.user = decoded.user;
13         next();
14     } catch (err) {
15         res.status(401).json({ msg: 'Token no es válido' });
16     }
17 };
18

```

Ilustración 41: Autenticación

4.19.2.2 Descripción del Código

Importación del Paquete JWT:

`const jwt = require('jsonwebtoken');`: Se utiliza para trabajar con JSON Web Tokens.

Exportación del Middleware:

`module.exports = function (req, res, next) {...}`: Define y exporta el middleware de autenticación como una función.

Obtención del Token:

`const token = req.header('x-auth-token');`: Obtiene el token JWT de la cabecera de la solicitud con el nombre x-auth-token.

Verificación de la Presencia del Token:

Si no se encuentra un token en la cabecera, se devuelve una respuesta de error 401 (no autorizado) con un mensaje indicando la ausencia del token.

Verificación del Token:

`const decoded = jwt.verify(token, process.env.JWT_SECRET);`; Verifica la validez del token utilizando el secreto JWT definido en las variables de entorno.

Si el token es válido, se decodifica y se adjunta la información del usuario (`decoded.user`) al objeto `req`, permitiendo que los controladores posteriores accedan a ella.

Manejo de Errores:

Si el token no es válido, se captura el error y se devuelve una respuesta de error 401 (no autorizado) con un mensaje indicando que el token no es válido.

4.19.2.3 Importancia

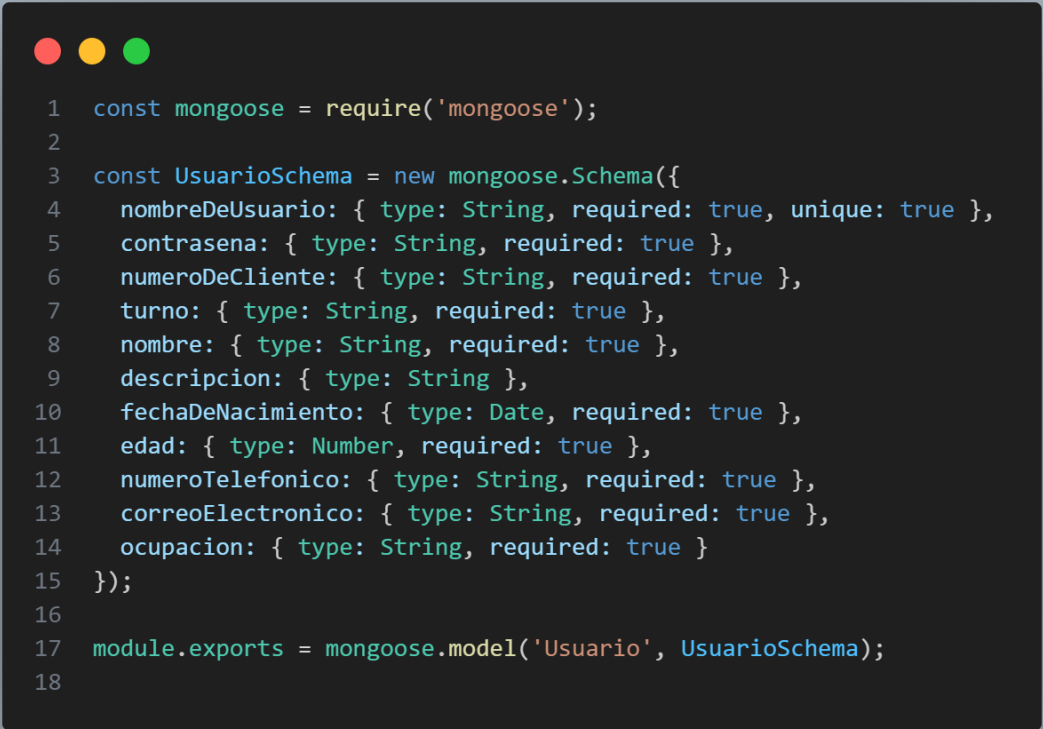
Este middleware es esencial para proteger las rutas de la aplicación, asegurando que solo los usuarios autenticados puedan acceder a ciertas funcionalidades. Al verificar la validez del token JWT en cada solicitud, se garantiza que las operaciones sensibles estén protegidas contra accesos no autorizados, mejorando así la seguridad general del sistema.

4.19.3 Backend / Esquema de Usuario

En esta sección, se describe la implementación del esquema de usuario en el backend del proyecto. El esquema de usuario define la estructura de los documentos de usuario en la base de datos MongoDB, estableciendo los campos necesarios y sus tipos de datos. En una aplicación que maneja la información de usuarios, es crucial definir un esquema claro y consistente para los documentos de usuario en la base de datos. MongoDB, en combinación con Mongoose, permite definir esquemas que estructuran los datos y aplican validaciones, asegurando la integridad y coherencia de la información almacenada.

4.19.3.1 Código / Esquema de Usuario

El siguiente código muestra la implementación del esquema de usuario utilizando Mongoose, una biblioteca de modelado de datos de MongoDB para Node.js.



```

1  const mongoose = require('mongoose');
2
3  const UsuarioSchema = new mongoose.Schema({
4    nombreDeUsuario: { type: String, required: true, unique: true },
5    contrasena: { type: String, required: true },
6    numeroDeCliente: { type: String, required: true },
7    turno: { type: String, required: true },
8    nombre: { type: String, required: true },
9    descripcion: { type: String },
10   fechaDeNacimiento: { type: Date, required: true },
11   edad: { type: Number, required: true },
12   numeroTelefonico: { type: String, required: true },
13   correoElectronico: { type: String, required: true },
14   ocupacion: { type: String, required: true }
15 });
16
17 module.exports = mongoose.model('Usuario', UsuarioSchema);
18

```

Ilustración 42: Modulos Usuario

4.19.3.2 Descripción del Código

Importación de Mongoose:

`const mongoose = require('mongoose');`: Importa la biblioteca Mongoose para interactuar con MongoDB.

Definición del Esquema de Usuario:

`const UsuarioSchema = new mongoose.Schema({...});`: Define un nuevo esquema para el modelo de usuario.

Campos del Esquema:

`nombreDeUsuario`: Un campo de tipo cadena de texto que es único y requerido. Almacena el nombre de usuario.

`contrasena`: Un campo de tipo cadena de texto requerido. Almacena la contraseña del usuario.

numeroDeCliente: Un campo de tipo cadena de texto requerido. Almacena el número de cliente del usuario.

turno: Un campo de tipo cadena de texto requerido. Almacena el turno del usuario.

nombre: Un campo de tipo cadena de texto requerido. Almacena el nombre del usuario.

descripcion: Un campo de tipo cadena de texto opcional. Almacena una descripción del usuario.

fechaDeNacimiento: Un campo de tipo fecha requerido. Almacena la fecha de nacimiento del usuario.

edad: Un campo de tipo número requerido. Almacena la edad del usuario.

numeroTelefonico: Un campo de tipo cadena de texto requerido. Almacena el número telefónico del usuario.

correoElectronico: Un campo de tipo cadena de texto requerido. Almacena el correo electrónico del usuario.

ocupacion: Un campo de tipo cadena de texto requerido. Almacena la ocupación del usuario.

Exportación del Modelo de Usuario:

`module.exports = mongoose.model('Usuario', UsuarioSchema);`; Crea y exporta el modelo de usuario basado en el esquema definido.

4.19.3.3 Importancia

El esquema de usuario es fundamental para estructurar y validar los datos de los usuarios en la base de datos. Definir un esquema claro y consistente ayuda a mantener la integridad de los datos, evitando inconsistencias y errores. Además, Mongoose proporciona capacidades de validación y control sobre los datos, lo que facilita el manejo de la información en la aplicación.

4.19.4 Backend / Esquema de Trabajador

En esta sección, se describe la implementación del esquema de trabajador en el backend del proyecto. El esquema de trabajador define la estructura de los documentos de trabajador en la base de datos MongoDB, estableciendo los campos necesarios y sus tipos de datos. Para gestionar la información de los trabajadores en una aplicación, es esencial definir un esquema claro y consistente que estructure los documentos de trabajador en la base de datos. MongoDB, junto con Mongoose,

facilita la definición de esquemas que estructuran los datos y aplican validaciones, asegurando la integridad y coherencia de la información almacenada.

4.19.4.1 Código / Esquema de Trabajador

El siguiente código muestra la implementación del esquema de trabajador utilizando Mongoose, una biblioteca de modelado de datos de MongoDB para Node.js.



```
1 // models/Trabajador.js
2 const mongoose = require('mongoose');
3
4 const TrabajadorSchema = new mongoose.Schema({
5   nombreDeUsuario: { type: String, required: true, unique: true },
6   contraseña: { type: String, required: true },
7 });
8
9 module.exports = mongoose.model('Trabajador', TrabajadorSchema);
10
```

Ilustración 43: Modulos Trabajadores

4.19.4.2 Descripción del Código

Importación de Mongoose:

`const mongoose = require('mongoose');`: Importa la biblioteca Mongoose para interactuar con MongoDB.

Definición del Esquema de Trabajador:

`const TrabajadorSchema = new mongoose.Schema({...});`: Define un nuevo esquema para el modelo de trabajador.

Campos del Esquema:

`nombreDeUsuario`: Un campo de tipo cadena de texto que es único y requerido. Almacena el nombre de usuario del trabajador.

`contraseña`: Un campo de tipo cadena de texto requerido. Almacena la contraseña del trabajador.

Exportación del Modelo de Trabajador:

`module.exports = mongoose.model('Trabajador', TrabajadorSchema);` Crea y exporta el modelo de trabajador basado en el esquema definido.

4.19.4.3 Importancia

El esquema de trabajador es crucial para estructurar y validar los datos de los trabajadores en la base de datos. Definir un esquema claro y consistente ayuda a mantener la integridad de los datos, evitando inconsistencias y errores. Además, Mongoose proporciona capacidades de validación y control sobre los datos, lo que facilita el manejo de la información en la aplicación.

4.19.5 Backend / Rutas para la Gestión de Usuarios

En esta sección, se describe la implementación de las rutas para la gestión de usuarios en el backend del proyecto. Estas rutas permiten registrar nuevos usuarios, iniciar sesión y buscar usuarios en la base de datos. Utilizando Express, bcrypt y jsonwebtoken, se asegura una gestión segura y eficiente de los usuarios. Para gestionar de manera efectiva la información de los usuarios en el proyecto, es fundamental contar con un conjunto de rutas bien definidas que permitan realizar operaciones básicas como el registro, inicio de sesión y búsqueda de usuarios. En este contexto, se utiliza Express para definir las rutas del servidor, bcrypt para la encriptación de contraseñas y jsonwebtoken para la autenticación basada en tokens.

4.19.5.1 Código / Rutas para la Gestión de Usuarios

El siguiente código muestra la implementación de las rutas para la gestión de usuarios utilizando Express

```

1  const express = require('express');
2  const router = express.Router();
3  const bcrypt = require('bcryptjs');
4  const Usuario = require('../models/Usuario');
5  const jwt = require('jsonwebtoken');
6
7  // Registro de usuario
8  router.post('/registrar', async (req, res) => {
9    const { nombreDeUsuario, contrasena, numeroDeCliente, turno, nombre, descripcion, fechaDeNacimiento, edad, numeroTelefonico, correoElectronico, ocupacion } = req.body;
10
11    try {
12      let usuario = await Usuario.findOne({ nombreDeUsuario });
13      if (usuario) {
14        return res.status(400).json({ msg: 'Usuario ya existe' });
15      }
16
17      // Validar campos obligatorios
18      if (!nombreDeUsuario || !contrasena || !numeroDeCliente || !turno || !nombre || !fechaDeNacimiento || !edad || !numeroTelefonico || !correoElectronico || !ocupacion) {
19        return res.status(400).json({ msg: 'Faltan campos obligatorios' });
20      }
21
22      // Crear nuevo usuario
23      usuario = new Usuario({
24        nombreDeUsuario,
25        contrasena: await bcrypt.hash(contrasena, 10),
26        numeroDeCliente,
27        turno,
28        nombre,
29        descripcion,
30        fechaDeNacimiento,
31        edad,
32        numeroTelefonico,
33        correoElectronico,
34        ocupacion
35      });
36
37      await usuario.save();
38      res.status(201).json({ msg: 'Usuario registrado exitosamente' });
39    } catch (err) {
40      console.error('Error en el servidor:', err);
41      res.status(500).send('Error en el servidor');
42    }
43  });
44
45  // Inicio de sesión de usuario
46  router.post('/iniciar-sesion', async (req, res) => {
47    const { nombreDeUsuario, contrasena, numeroDeCliente } = req.body;
48
49    try {
50      // Buscar usuario por nombre de usuario y número de cliente
51      let usuario = await Usuario.findOne({ nombreDeUsuario, numeroDeCliente });
52      if (!usuario) {
53        return res.status(400).json({ msg: 'Credenciales incorrectas' });
54      }
55
56      // Verificar la contraseña
57      const esCoincidente = await bcrypt.compare(contrasena, usuario.contrasena);
58      if (!esCoincidente) {
59        return res.status(400).json({ msg: 'Credenciales incorrectas' });
60      }
61
62      // Crear y enviar token JWT
63      const token = jwt.sign({ id: usuario._id }, process.env.JWT_SECRET, { expiresIn: '1h' });
64      res.json({ token });
65    } catch (err) {
66      console.error('Error en el servidor:', err);
67      res.status(500).send('Error en el servidor');
68    }
69  });
70
71  // Ruta de búsqueda
72  router.get('/buscar', async (req, res) => {
73    const { nombre, nombreDeUsuario, numeroDeCliente } = req.query;
74
75    try {
76      const query = {};
77      if (nombre) query.nombre = nombre;
78      if (nombreDeUsuario) query.nombreDeUsuario = nombreDeUsuario;
79      if (numeroDeCliente) query.numeroDeCliente = numeroDeCliente;
80
81      const usuarios = await Usuario.find(query);
82      res.json(usuarios);
83    } catch (err) {
84      console.error('Error en la búsqueda:', err);
85      res.status(500).json({ msg: 'Error en el servidor' });
86    }
87  });
88
89  module.exports = router;
90
91
92

```

Ilustración 44: Rutas de Usuarios

4.19.5.2 Descripción del Código

Importación de Dependencias:

`const express = require('express');`; Importa el framework Express para crear rutas.

`const bcrypt = require('bcryptjs');`; Importa bcrypt para encriptar contraseñas.

`const Usuario = require('../models/Usuario');`; Importa el modelo de usuario.

`const jwt = require('jsonwebtoken');`; Importa jsonwebtoken para la generación de tokens JWT.

Registro de Usuario:

Ruta: `router.post('/registrar', async (req, res) => {...});`

Verifica si el nombre de usuario ya existe.

Valida que todos los campos obligatorios estén presentes.

Encripta la contraseña antes de guardar el nuevo usuario en la base de datos.

Responde con un mensaje de éxito o error según corresponda.

Inicio de Sesión de Usuario:

Ruta: `router.post('/iniciar-sesion', async (req, res) => {...});`

Verifica la existencia del usuario mediante el nombre de usuario y el número de cliente.

Compara la contraseña ingresada con la almacenada en la base de datos.

Genera un token JWT si las credenciales son correctas.

Ruta de Búsqueda de Usuarios:

Ruta: `router.get('/buscar', async (req, res) => {...});`

Permite buscar usuarios por nombre, nombre de usuario o número de cliente.

Retorna los usuarios que coinciden con los criterios de búsqueda.

4.19.5.3 Importancia

Definir rutas claras y seguras para la gestión de usuarios es esencial para cualquier aplicación que maneje información sensible. La combinación de Express, bcrypt y jsonwebtoken proporciona una base sólida para la autenticación y autorización de usuarios, garantizando que la información se maneje de manera segura y eficiente.

4.19.6 Backend / Rutas para la Gestión de Trabajadores

En esta sección, se detalla la implementación de las rutas para la gestión de trabajadores en el backend del proyecto. Estas rutas permiten registrar nuevos trabajadores, iniciar sesión y obtener una lista de todos los trabajadores. Utilizando Express, bcrypt, jsonwebtoken y express-validator, se asegura una gestión segura y eficiente de los trabajadores. Para gestionar de manera efectiva la información de los trabajadores en una aplicación, es fundamental contar con un conjunto de rutas bien definidas que permitan realizar operaciones básicas como el registro, inicio de sesión y obtención de trabajadores. En este contexto, se utiliza Express para definir las rutas del servidor, bcrypt para la encriptación de contraseñas, jsonwebtoken para la autenticación basada en tokens y express-validator para la validación de datos.

4.19.6.1 Código / Rutas para la Gestión de Trabajadores

El siguiente código muestra la implementación de las rutas para la gestión de trabajadores utilizando Express.

4.19.6.2 Descripción del Código

Importación de Dependencias:

`const express = require('express');` Importa el framework Express para crear rutas.

`const bcrypt = require('bcryptjs');` Importa bcrypt para encriptar contraseñas.

`const { check, validationResult } = require('express-validator');` Importa express-validator para la validación de datos.

`const Trabajador = require('../models/Trabajador');` Importa el modelo de trabajador.

`const jwt = require('jsonwebtoken');` Importa jsonwebtoken para la generación de tokens JWT.

`const auth = require('../middleware/auth');` Importa el middleware de autenticación.

Registro de Trabajador:

Ruta: `router.post('/registrar', [...], async (req, res, next) => {...});`

Valida los campos de nombre de usuario y contraseña.

Verifica si el nombre de usuario ya existe.

Encripta la contraseña antes de guardar el nuevo trabajador en la base de datos.

Responde con un mensaje de éxito o error según corresponda.

Inicio de Sesión de Trabajador:

Ruta: `router.post('/login', [...], async (req, res, next) => {...});`

Valida los campos de nombre de usuario y contraseña.

Verifica la existencia del trabajador mediante el nombre de usuario.

Compara la contraseña ingresada con la almacenada en la base de datos.

Genera un token JWT si las credenciales son correctas.

Obtener Todos los Trabajadores:

Ruta: `router.get('/', auth, async (req, res, next) => {...});`

Utiliza el middleware de autenticación para proteger la ruta.

Obtiene y devuelve la lista de todos los trabajadores almacenados en la base de datos.

4.19.6.3 Importancia

Definir rutas claras y seguras para la gestión de trabajadores es esencial para cualquier aplicación que maneje información sensible. La combinación de Express, bcrypt, jsonwebtoken y express-validator proporciona una base sólida para la autenticación y autorización de trabajadores, garantizando que la información se maneje de manera segura y eficiente.

```

1  const express = require('express');
2  const router = express.Router();
3  const bcrypt = require('bcryptjs');
4  const { check, validationResult } = require('express-validator');
5  const Trabajador = require('../models/Trabajador');
6  const jwt = require('jsonwebtoken');
7  const auth = require('../middleware/auth');
8
9  // Registro de trabajador
10 router.post(
11   '/registrar',
12   [
13     check('nombreDeUsuario', 'Nombre de usuario es requerido').not().isEmpty(),
14     check('contrasena', 'Contraseña debe tener mínimo 6 caracteres').isLength({ min: 6 })
15   ],
16   async (req, res, next) => {
17     const errors = validationResult(req);
18     if (!errors.isEmpty()) {
19       return res.status(400).json({ errors: errors.array() });
20     }
21     const { nombreDeUsuario, contrasena } = req.body;
22     try {
23       let trabajador = await Trabajador.findOne({ nombreDeUsuario });
24       if (trabajador) {
25         return res.status(400).json({ msg: 'Trabajador ya existe' });
26       }
27       trabajador = new Trabajador({
28         nombreDeUsuario,
29         contrasena: await bcrypt.hash(contrasena, 10),
30       });
31       await trabajador.save();
32       res.status(201).json({ msg: 'Trabajador registrado' });
33     } catch (err) {
34       next(err);
35     }
36   }
37 );
38
39 // Inicio de sesión de trabajador
40 router.post(
41   '/login',
42   [
43     check('nombreDeUsuario', 'Nombre de usuario es requerido').not().isEmpty(),
44     check('contrasena', 'Contraseña es requerida').exists()
45   ],
46   async (req, res, next) => {
47     const errors = validationResult(req);
48     if (!errors.isEmpty()) {
49       return res.status(400).json({ errors: errors.array() });
50     }
51     const { nombreDeUsuario, contrasena } = req.body;
52     try {
53       const trabajador = await Trabajador.findOne({ nombreDeUsuario });
54       if (!trabajador) {
55         return res.status(400).json({ msg: 'Credenciales incorrectas' });
56       }
57       const esCoincidente = await bcrypt.compare(contrasena, trabajador.contrasena);
58       if (!esCoincidente) {
59         return res.status(400).json({ msg: 'Credenciales incorrectas' });
60       }
61       const token = jwt.sign({ id: trabajador._id }, process.env.JWT_SECRET, { expiresIn: '1h' });
62       res.json({ token });
63     } catch (err) {
64       next(err);
65     }
66   }
67 );
68
69 // Obtener todos los trabajadores
70 router.get('/', auth, async (req, res, next) => {
71   try {
72     const trabajadores = await Trabajador.find();
73     res.json(trabajadores);
74   } catch (err) {
75     next(err);
76   }
77 });
78
79 module.exports = router;
81

```

Ilustración 45: Rutas de Trabajadores

4.19.7 Backend / Servidor Principal

En esta sección, se describe la configuración principal del servidor y la definición de las rutas para los usuarios y trabajadores en el backend del proyecto. Utilizando Express, dotenv, cors y un middleware personalizado para el manejo de errores, se configura un entorno robusto y seguro para la aplicación.

La configuración del servidor es una parte fundamental en el desarrollo de una aplicación backend. Utilizando Express, se puede definir la estructura y los middlewares necesarios para manejar solicitudes y respuestas de manera eficiente. Además, la integración con dotenv permite gestionar variables de entorno, mientras que cors facilita la comunicación entre el frontend y el backend. Un middleware personalizado para el manejo de errores garantiza que las respuestas sean coherentes y útiles en caso de fallos.

4.19.7.1 Código / Configuración del Servidor y Rutas

El siguiente código muestra cómo configurar el servidor principal y definir las rutas para usuarios y trabajadores en el backend del proyecto.

Descripción del Código

1. Importación de Dependencias:

- `const express = require('express');`: Importa el framework Express para crear y configurar el servidor.
- `const connectDB = require('./config/db');`: Importa la función para conectar a la base de datos.
- `const dotenv = require('dotenv');`: Importa dotenv para manejar variables de entorno.
- `const cors = require('cors');`: Importa cors para permitir solicitudes de diferentes dominios.
- `const errorHandler = require('./middleware/errorHandler');`: Importa un middleware personalizado para el manejo de errores.

2. Configuración del Entorno:

- `dotenv.config();`: Carga las variables de entorno desde un archivo `.env`.
- `connectDB();`: Conecta a la base de datos utilizando la configuración de la base de datos.

3. Configuración de Express:

- `const app = express();`: Crea una instancia de la aplicación Express.
- `app.use(cors());`: Configura CORS para permitir solicitudes de diferentes dominios.
- `app.use(express.json());`: Añade middleware para parsear solicitudes con cuerpo en formato JSON.

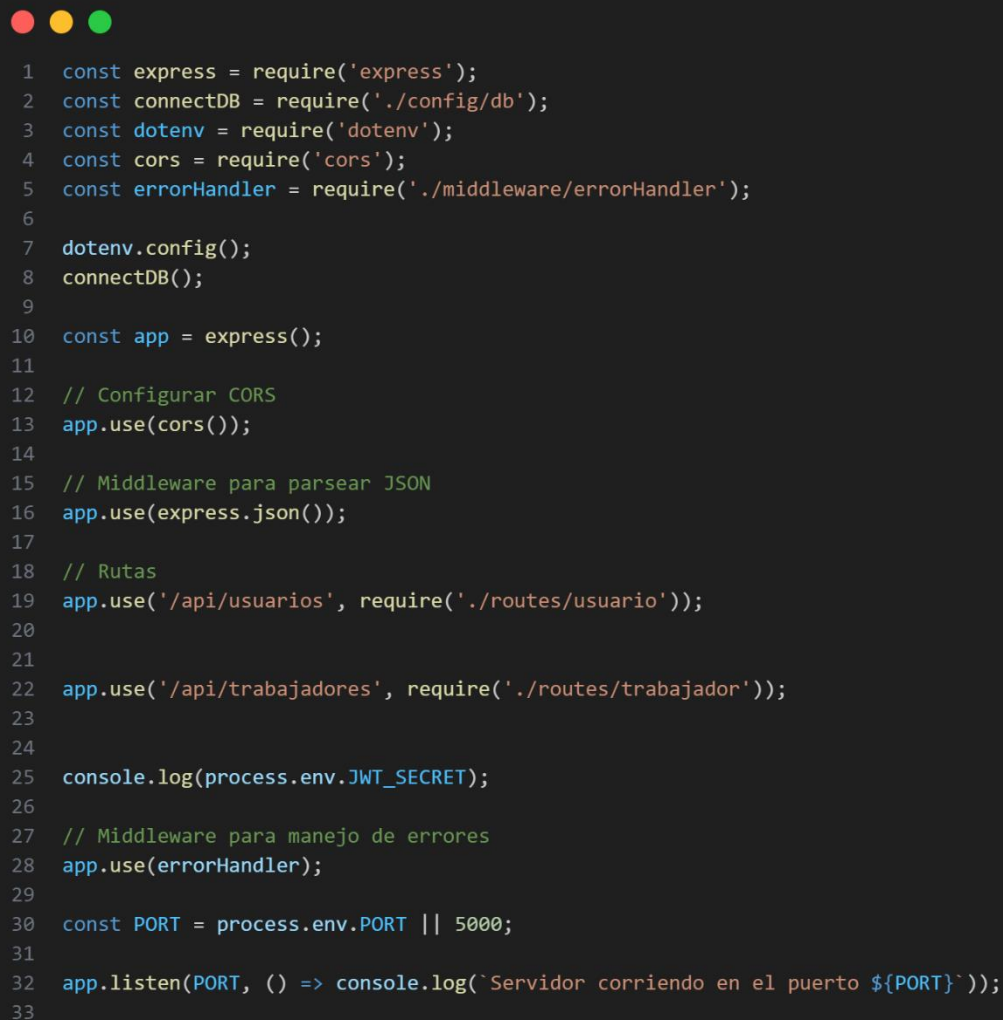
4. Definición de Rutas:

- `app.use('/api/usuarios', require('./routes/usuario'));`: Define la ruta para gestionar usuarios.

- `app.use('/api/trabajadores', require('./routes/trabajador'))`:: Define la ruta para gestionar trabajadores.
- 5. **Impresión de la Variable de Entorno:**
 - `console.log(process.env.JWT_SECRET)`:: Imprime en consola el valor de la variable de entorno `JWT_SECRET` para verificación.
- 6. **Manejo de Errores:**
 - `app.use(errorHandler)`:: Añade un middleware personalizado para el manejo de errores.
- 7. **Iniciar el Servidor:**
 - `const PORT = process.env.PORT || 5000`:: Define el puerto en el que correrá el servidor.
 - `app.listen(PORT, () => console.log(Servidor corriendo en el puerto ${PORT}))`:: Inicia el servidor y muestra un mensaje en consola indicando el puerto en el que está corriendo.

4.19.7.2 Importancia

Configurar adecuadamente el servidor y definir las rutas de manera clara y estructurada es esencial para el desarrollo de cualquier aplicación backend. Utilizando Express y complementándolo con `dotenv`, `cors` y un middleware personalizado para el manejo de errores, se garantiza un entorno de desarrollo seguro y eficiente, facilitando la integración con el frontend y mejorando la experiencia de los desarrolladores y usuarios finales.



```
1  const express = require('express');
2  const connectDB = require('./config/db');
3  const dotenv = require('dotenv');
4  const cors = require('cors');
5  const errorHandler = require('./middleware/errorHandler');
6
7  dotenv.config();
8  connectDB();
9
10 const app = express();
11
12 // Configurar CORS
13 app.use(cors());
14
15 // Middleware para parsear JSON
16 app.use(express.json());
17
18 // Rutas
19 app.use('/api/usuarios', require('./routes/usuario'));
20
21
22 app.use('/api/trabajadores', require('./routes/trabajador'));
23
24
25 console.log(process.env.JWT_SECRET);
26
27 // Middleware para manejo de errores
28 app.use(errorHandler);
29
30 const PORT = process.env.PORT || 5000;
31
32 app.listen(PORT, () => console.log(`Servidor corriendo en el puerto ${PORT}`));
33
```

Ilustración 46: Servidor Node

5 Control

5.1 Pruebas y Depuración

Se realizaron pruebas exhaustivas en la aplicación web utilizando Postman para garantizar el correcto funcionamiento de las rutas y funcionalidades del backend. Se identificaron y corrigieron errores en la autenticación, manejo de datos y comunicación entre el frontend y el backend. Además, se realizaron pruebas de rendimiento para asegurar una experiencia de usuario fluida y eficiente.

5.2 Optimización y Ajustes Finales

Se realizaron ajustes finales en el diseño y funcionalidad de la aplicación, con un enfoque en la optimización de los tiempos de carga y la accesibilidad. Se llevaron a cabo pruebas de usabilidad para ajustar cualquier aspecto que pudiera mejorar la experiencia del usuario, asegurando que la aplicación sea intuitiva y eficiente.

6 Cierre

6.1 Lanzamiento y Despliegue

Una vez completadas las pruebas y ajustes finales, se procedió al lanzamiento y despliegue de la aplicación web en el servidor de producción utilizando Hostinger. Se verificó que todas las funcionalidades operaran correctamente en el entorno en vivo, garantizando una transición suave antes de presentar oficialmente la aplicación a los usuarios.

6.2 Repositorio de GitHub



Frontend:

<https://github.com/theLuis77/backside-Frontend-.git>



Bakende:

<https://github.com/theLuis77/BacksideSiluet-Bakende-.git>